

COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms	SECTION: CPE21S1
DATE PERFORMED: September 8, 2026	DATE SUBMITTED: September 8, 2026
NAME: Mendoza, Roy Gabriel I.	INSTRUCTOR: Engr. Jimlord Quejado
ACTIVITY NO. 7 SORTING ALGORITHMS: BUBBLE, SELECTION, AND INSERTION SORT	
1. PROCEDURE OUTPUT	

Code Console Screenshot	CODE	<pre> 1 > //... 4 5 #include <iostream> 6 #include <cstdlib> 7 #include <time.h> 8 #include "ALGO.h" 9 10 const int max_size = 100; 11 12 void showArr(const int arr[], int size); 13 14 int main() { 15 int original[max_size]; 16 int arrBubble[max_size]; 17 int arrInsertion[max_size]; 18 int arrSelection[max_size]; 19 20 // Seed the random number generator 21 srand(Seed: time(0)); 22 23 // 1. Generate random values for the master array and copy them 24 for (int i = 0; i < max_size; i++) { 25 original[i] = rand() % 100; 26 27 // Copy to separate the arrays 28 arrBubble[i] = original[i]; 29 arrInsertion[i] = original[i]; 30 arrSelection[i] = original[i]; 31 } 32 33 // Display original unsorted array 34 std::cout << "===== ORIGINAL ARRAY =====\n"; 35 showArr(original, max_size); 36 std::cout << "\n"; </pre>
	OUTPUT	<pre> "C:\Users\royga\CLionProjects\untitled1\MSA App\Main.exe" ===== ORIGINAL ARRAY ===== 89 35 94 90 22 94 74 21 41 82 43 49 91 2 20 79 47 54 23 44 96 49 22 2 92 32 47 91 48 91 53 45 14 11 31 55 29 98 51 67 54 19 90 29 38 66 97 29 64 73 68 60 45 29 91 13 57 45 63 27 24 97 31 21 72 81 75 49 28 75 32 68 81 89 46 58 33 88 82 1 58 22 17 4 91 87 53 35 92 51 32 16 58 41 2 95 26 18 14 25 ===== BUBBLE SORT ===== </pre>

Observations	From my observation, this part of the code generates the dataset of random integers but is limited only to integer of 100 due to the %100. This is the base of the 3 sortation for it generates the data set that they use. As for the Time complexity, It would be O(n) because the for loop would need to run entirely to produce the 100 element dataset or any amount that we will limit into.
--------------	--

Table 7-1. Array of Values for Sort Algorithm Testing

Code Console Screenshot	CODE <pre> 1 // 2 // Created by povga on 9/8/2026. 3 // 4 5 #ifndef UNTITLED1_ALG0_H 6 #define UNTITLED1_ALG0_H 7 #include <bits/stdc++.h> 8 9 template <typename T> 10 void bubbleSort(T arr[], int arrSize) { 11 for (int i=0; i<arrSize-1; i++) { 12 for (int j=i+1; j<arrSize; j++) { 13 if (arr[j] > arr[i]) { //here can change if descending or ascending depending if < or > 14 std::swap(arr[j], arr[i]); 15 } 16 } 17 } 18 } </pre> <pre> 37 38 // --- BUBBLE SORT --- 39 std::cout << "===== BUBBLE SORT =====\n"; 40 bubbleSort(arrBubble, max_size); 41 showArr(arrBubble, max_size); 42 std::cout << "\n"; </pre> OUTPUT <pre> "C:\Users\nguel\OneDrive\Projects\untitled1\main.exe" ===== ORIGINAL ARRAY ===== 80 35 94 90 52 94 74 31 41 82 43 49 91 3 38 79 47 54 23 44 96 68 12 3 92 32 47 91 48 93 53 45 14 11 31 55 29 98 51 67 54 19 98 29 18 66 97 29 64 73 68 60 65 29 91 13 57 45 63 27 24 97 31 21 72 81 75 49 20 79 12 68 81 89 66 58 33 88 82 1 58 22 17 4 91 87 33 15 92 51 32 16 18 61 2 95 26 18 14 25 ===== BUBBLE SORT ===== 98 97 97 96 95 94 94 92 92 91 91 91 91 90 89 88 87 82 81 81 80 79 75 74 73 72 68 68 67 67 66 65 64 63 61 60 58 58 57 56 55 54 53 52 51 51 49 49 48 47 46 45 44 43 41 35 33 33 32 32 31 31 31 28 29 29 29 27 26 25 24 23 23 21 20 19 18 18 17 16 15 14 14 12 12 12 11 10 4 3 2 1 </pre>
Observations	From my observation and understanding, Bubble sort Compares the adjacent elements across the array, and pushes or swaps the higher value to the right side until all is sorted ascending or descending/ in the correct position. As for the time complexity, it would be O(n²) on the best and worst, because it would need to compare each element of the array until the end even if it is correct.

Table 7-2. Bubble Sort Technique

Code Console Screenshot	
-------------------------	--

	CODE	<pre>36 37 template <typename T> 38 int routine_smallest(const T A[], int max_size, int k) { 39 int position = k; // Initialize position to k in case A[k] is already the smallest 40 T smallestElement = A[k]; 41 42 for (int j = k + 1; j < max_size; j++) { 43 if (A[j] < smallestElement) { 44 smallestElement = A[j]; 45 position = j; 46 } 47 } 48 return position; 49 } 50 51 template <typename T> 52 void selectionSort(T arr[], int arrSize) { 53 for (int i = 0; i < arrSize - 1; i++) { 54 int pos = routine_smallest(arr, arrSize, i); 55 56 // Clean swap using T instead of int 57 std::swap(arr[i], arr[pos]); 58 } 59 } 60 #endif //UNTITLED1_ALGO_H</pre>
	OUTPUT	<pre>***** ORIGINAL ARRAY ***** 80 35 94 90 52 94 74 33 43 82 49 91 3 38 79 47 56 23 44 96 68 12 3 92 32 67 93 48 93 55 45 14 11 31 55 29 98 51 67 54 19 98 29 38 66 97 29 64 73 68 66 65 29 91 13 57 45 63 27 24 97 31 21 72 83 75 49 28 75 12 68 81 89 46 58 33 88 82 1 58 22 17 4 91 67 33 15 92 31 32 16 18 61 2 95 26 18 14 25 ***** SELECTION SORT ***** 1 2 3 4 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 29 29 29 30 31 31 31 32 33 33 35 41 43 44 45 46 47 48 49 49 51 51 52 53 54 55 56 57 58 58 60 61 63 64 65 66 67 67 68 68 68 68 72 73 74 75 75 79 80 81 81 82 82 87 88 89 90 90 91 91 91 92 92 92 94 94 95 96 97 97 98</pre>
Observations	From my understanding, Selection sort divides the array into the left sorted and right unsorted regions, with the helper function of routine_smallest that scans the unsorted region to find the exact index of the minimum element. As for the time complexity, it would O(n^2) because it uses an outer loop to set the index and calls the helper function which also is looping.	

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	
---------------------------	--

	<table border="1"> <tr> <td data-bbox="428 201 570 940">CODE</td><td data-bbox="570 201 1416 940"> <pre> 20 template <typename T> 21 void insertionSort(T arr[], int arrSize) { 22 int k=0, j; 23 T temp; 24 25 while (k<arrSize){ 26 temp=arr[k]; 27 j = k-1; 28 while (j>=0 && temp < arr[j]) { 29 arr[j+1]=arr[j]; 30 j--; 31 } 32 arr[j+1]= temp; 33 k++; 34 } 35 } </pre> </td></tr> <tr> <td data-bbox="428 940 570 1150">OUTPUT</td><td data-bbox="570 940 1416 1150"> <pre> "C:\Users\yuyga\CLionProjects\untitled1\H08_H01\Main.exe" ===== INSERTION SORT ===== 80 55 94 90 52 94 74 31 41 82 43 49 91 1 30 79 47 56 23 44 96 68 12 3 92 32 67 91 48 91 53 45 14 11 31 55 29 98 51 67 54 19 90 29 18 66 97 29 64 73 68 60 65 29 91 13 57 45 63 27 26 97 31 21 72 81 75 49 20 75 12 68 81 89 46 58 33 88 82 1 58 22 17 4 91 87 33 15 92 51 32 16 18 61 2 95 26 10 14 20 ===== INSERTION SORT ===== 1 2 3 4 10 11 12 12 13 14 14 15 16 17 18 18 19 20 21 22 23 24 25 26 27 29 29 29 38 31 31 31 32 32 33 33 35 41 43 44 45 45 46 47 48 49 49 51 51 52 53 54 55 56 57 58 58 60 61 63 64 65 66 67 67 68 68 68 72 73 74 75 75 79 80 81 81 82 82 82 87 88 89 90 90 91 91 91 91 92 92 94 94 95 96 97 97 98 </pre> </td></tr> </table>	CODE	<pre> 20 template <typename T> 21 void insertionSort(T arr[], int arrSize) { 22 int k=0, j; 23 T temp; 24 25 while (k<arrSize){ 26 temp=arr[k]; 27 j = k-1; 28 while (j>=0 && temp < arr[j]) { 29 arr[j+1]=arr[j]; 30 j--; 31 } 32 arr[j+1]= temp; 33 k++; 34 } 35 } </pre>	OUTPUT	<pre> "C:\Users\yuyga\CLionProjects\untitled1\H08_H01\Main.exe" ===== INSERTION SORT ===== 80 55 94 90 52 94 74 31 41 82 43 49 91 1 30 79 47 56 23 44 96 68 12 3 92 32 67 91 48 91 53 45 14 11 31 55 29 98 51 67 54 19 90 29 18 66 97 29 64 73 68 60 65 29 91 13 57 45 63 27 26 97 31 21 72 81 75 49 20 75 12 68 81 89 46 58 33 88 82 1 58 22 17 4 91 87 33 15 92 51 32 16 18 61 2 95 26 10 14 20 ===== INSERTION SORT ===== 1 2 3 4 10 11 12 12 13 14 14 15 16 17 18 18 19 20 21 22 23 24 25 26 27 29 29 29 38 31 31 31 32 32 33 33 35 41 43 44 45 45 46 47 48 49 49 51 51 52 53 54 55 56 57 58 58 60 61 63 64 65 66 67 67 68 68 68 72 73 74 75 75 79 80 81 81 82 82 82 87 88 89 90 90 91 91 91 91 92 92 94 94 95 96 97 97 98 </pre>
CODE	<pre> 20 template <typename T> 21 void insertionSort(T arr[], int arrSize) { 22 int k=0, j; 23 T temp; 24 25 while (k<arrSize){ 26 temp=arr[k]; 27 j = k-1; 28 while (j>=0 && temp < arr[j]) { 29 arr[j+1]=arr[j]; 30 j--; 31 } 32 arr[j+1]= temp; 33 k++; 34 } 35 } </pre>				
OUTPUT	<pre> "C:\Users\yuyga\CLionProjects\untitled1\H08_H01\Main.exe" ===== INSERTION SORT ===== 80 55 94 90 52 94 74 31 41 82 43 49 91 1 30 79 47 56 23 44 96 68 12 3 92 32 67 91 48 91 53 45 14 11 31 55 29 98 51 67 54 19 90 29 18 66 97 29 64 73 68 60 65 29 91 13 57 45 63 27 26 97 31 21 72 81 75 49 20 75 12 68 81 89 46 58 33 88 82 1 58 22 17 4 91 87 33 15 92 51 32 16 18 61 2 95 26 10 14 20 ===== INSERTION SORT ===== 1 2 3 4 10 11 12 12 13 14 14 15 16 17 18 18 19 20 21 22 23 24 25 26 27 29 29 29 38 31 31 31 32 32 33 33 35 41 43 44 45 45 46 47 48 49 49 51 51 52 53 54 55 56 57 58 58 60 61 63 64 65 66 67 67 68 68 68 72 73 74 75 75 79 80 81 81 82 82 82 87 88 89 90 90 91 91 91 91 92 92 94 94 95 96 97 97 98 </pre>				
Observations	From my understanding, Insertion sort moves the large elements to one position, basically like building another array, and inserting the temp into the gap created by the shift. As for the Time Complexity, it would also be $O(n^2)$ since the outer loop also run and $n-1$ then another of n causing n^2., but the best case could actually be $O(n)$ if the array is already sorted.				

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

List of Candidates

Problem: Generate an array $A[0 \dots 100]$ of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

Deliverables:

- Pseudocode of Algorithm
- Screenshot of Algorithm Code
- Output Testing

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm

Question: Was your developed vote counting algorithm effective? Why or why not?

8. Conclusion

Provide the following:

- Summary of lessons learned
- Analysis of the procedure
- Analysis of the supplementary activity
- Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

PSEUDOCODE	<pre>ALGORITHM VoteCountingSystem(votes, N) // 1. Sort the array using Insertion Sort FOR i FROM 1 TO N - 1 DO key = votes[i], j = i - 1 WHILE j >= 0 AND votes[j] > key votes[j + 1] = votes[j], j = j - 1 END WHILE LOOP</pre>
-------------------	--

	<pre> votes[j + 1] = key END FOR LOOP // 2. Tally vote frequencies DECLARE voteCount[6] = {0} FOR i FROM 0 TO N - 1 DO voteCount[votes[i]]++ END FOR LOOP // 3. Find max votes and detect ties maxVotes = 0, winnerCount = 0 FOR id FROM 1 TO 5 DO IF voteCount[id] > maxVotes THEN maxVotes = voteCount[id] END FOR LOOP FOR id FROM 1 TO 5 DO IF voteCount[id] == maxVotes THEN winnerCount++ END FOR LOOP // 4. Output results PRINT (winnerCount == 1) ? "WINNER:" : "TIE:" FOR id FROM 1 TO 5 IF voteCount[id] == maxVotes THEN PRINT id, CandNames(id), maxVotes END IF END FOR LOOP END ALGORITHM</pre>
--	---

CODE

```
1  //
2  // Created by royga on 9/9/2026.
3  //
4  #include <iostream>
5  #include <string>
6  #include <time.h>
7  #include <cstdlib>
8  #include "ALGO.h"
9
10
11  //SHORTCUT COPY PASTE: [====][std::cout<<"\n\n";]
12  //constant number of votes
13  const int voters = 100;
14
15  //to show the array elements
16  void showArr(const int arr[], int size);
17
18  // function to show candidates number to their name
19  std::string CandNames(int candID) {
20      switch (candID) {
21          case 1:
22              return "Bo Dalton Capistrano";
23          case 2:
24              return "Cornelius Raymon Agustin";
25          case 3:
26              return "Deja Jayla Bañaga";
27          case 4:
28              return "Lalla Brielle Yabut";
29          case 5:
30              return "Franklin Relano Castro";
31          default:
32              return "UNKNOWN";
33      }
34  }
```

```

35 ▶ int main() {
36     int votes[voters];
37     //seed
38     srand(Seed: time(0));
39     //generate the 100 elements but limit it to only to numbers 1 to 5
40     for (int i = 0; i < voters; i++) {
41         votes[i] = (rand() % 5)+1; //cheating the generation to get the 5 instead of just 0
42     }
43
44     //show the unsorted array
45     std::cout<< "=====UNSORTED VOTES=====\n";
46     showArr(votes, size: voters);
47     std::cout<< "\n\n";
48
49     //sort using insertion sort
50     std::cout<< "=====INSERTION SORT=====\n";
51     insertionSort(votes, arrSize: voters);
52     std::cout<< "\n\n";
53
54     //show the sorted numbers
55     showArr(votes, size: voters);
56     std::cout<< "\n\n";
57
58     //tally the votes
59     int voteCount[6] = {0};
60     for (int i = 0; i < voters; i++) {
61         voteCount[votes[i]]++;
62     }
63
64     //display the tallied number of votes
65     std::cout<< "===== TALLY =====\n";
66     for (int i=1; i<=5; i++) {
67         std::cout<< "CANDIDATE"<<i<<": "<<CandNames(i)<<": "<<voteCount[i]<< " VOTES\n";
68     }
69     std::cout<< "\n\n";
70 }

```

```

71 //finding the highest vote count
72 int maxVotes = 0;
73 for (int i=1; i<=5; i++) {
74     if (voteCount[i] > maxVotes) {
75         maxVotes = voteCount[i];
76     }
77 }
78
79 //if in any case has ties(nah)
80 int winnerCount = 0;
81 for (int i=1; i<=5; i++) {
82     if (voteCount[i] == maxVotes) {
83         winnerCount++;
84     }
85 }
86
87 //Show the result
88 std::cout<< "===== WINNER =====\n";
89 if (winnerCount ==1) {
90     //for single win
91     for (int i=1; i<=5; i++) {
92         if (voteCount[i] == maxVotes) {
93             std::cout<< "\nCANDIDATE "<< i << " ("<< CandNames(i)<< ") with "<< maxVotes << " votes\n";
94         }
95     }
96 }else { //for the tie
97     std::cout<< "IT IS A TIE! The following "<< winnerCount << " candidates tied with "<< maxVotes << " votes each:\n";
98
99     for (int i = 1; i <= 5; i++) {
100         if (voteCount[i] == maxVotes) {
101             std::cout << " - Candidate " << i << " (" << CandNames(i) << ")\n";
102         }
103     }
104 }
105 return 0;
106 }
107

```



```

107
108 → void showArr(const int arr[], int size)
109     for (int i = 0; i < size; i++) {
110         std::cout << arr[i] << " ";
111     }
112     std::cout << std::endl;
113 }

```

ALGO.h

```

1 //
2 // Created by royga on 9/8/2026.
3 //
4
5 #ifndef UNTITLED1_ALGO_H
6 #define UNTITLED1_ALGO_H
7
8 template <typename T>
9 void bubbleSort(T arr[], int arrSize) {
10     for (int i=0; i<arrSize-1; i++) {
11         for (int j=i+1; j<arrSize; j++) {
12             if (arr[j] > arr[i]) { //here can change if descending or ascending depending if < or >
13                 std::swap(&arr[j], &arr[i]);
14             }
15         }
16     }
17 }
18
19 template <typename T>
20 void insertionSort(T arr[], int arrSize) {
21     int k=0, j;
22     T temp;
23
24     while (k<arrSize){
25         temp=arr[k];
26         j = k-1;
27         while (j>=0 && temp < arr[j]) {
28             arr[j+1]=arr[j];
29             j--;
30         }
31         arr[j+1]= temp;
32         k++;
33     }
34 }
35

```

```

36 template <typename T>
37 int routine_smallest(const T A[], int max_size, int k) {
38     int position = k; // Initialize position to k in case A[k] is already the smallest
39     T smallestElement = A[k];
40
41     for (int j = k + 1; j < max_size; j++) {
42         if (A[j] < smallestElement) {
43             smallestElement = A[j];
44             position = j;
45         }
46     }
47     return position;
48 }
49
50 template <typename T>
51 void selectionSort(T arr[], int arrSize) {
52     for (int i = 0; i < arrSize - 1; i++) {
53         int pos = routine_smallest(arr, arrSize, i);
54
55         // Clean swap using T instead of int
56         std::swap(arr[i], arr[pos]);
57     }
58 }
59 #endif //UNTITLED1_ALGO_H

```

CONSOLE OUTPUT

[illegible]**MANUAL COUNT**

23+22+19+20+16

100

() MC M+ M- MR

x! x² x³ x^y e^x π

1/x √ ∛ √^y Inv Rad

e ln lg sin cos tan

C × ÷ ⊗

7 8 9 -

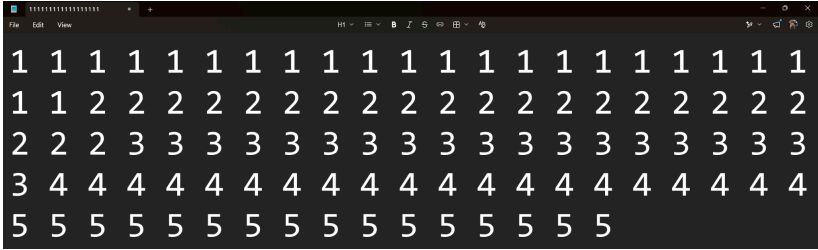
4 5 6 +

1 2 3 %

+/- 0 . =

From sorted:

Card 1:	23
2:	22
3:	19
4:	20
5:	16

	
COUNT RESULT ALGORITHM	<pre> ===== TALLY ===== CANDIDATE1: Bo Dalton Capistrano: 23 VOTES CANDIDATE2: Cornelius Raymon Agustin: 22 VOTES CANDIDATE3: Deja Jayla Bañaga: 19 VOTES CANDIDATE4: Lalla Brielle Yabut: 20 VOTES CANDIDATE5: Franklin Relano Castro: 16 VOTES </pre>
QUESTION's ANSWER	I would say yes because the created code combined a linear time tallying and the sorting (insertion) to create the tally that is accurate.

CONCLUSION:	
From this activity, I have learned the use of the sorting algorithms and how they are implemented(the bubble, insertion and the selection sort). In the procedure I have learned how to create a random number using the time.h and cstdlib libraries, also identified what are the time complexity of each of the sorting algorithms, for example with the Bubble sort, it has the worst of the 3 with the $O(n^2)$, then we have the insertion sort which can be used on repeating values and linear case with a complexity of $O(n)$. As for the Supplementary, It is just like a combination of the procedure, but I only implemented the Insertion sort for it is the best, as far as I know and how I implemented it. Then speaking of how I determine the candidate, I just use a Switch Case to say who wins for that exact number that we have on the voters. As for my feedback, maybe I should study the other sorting algos that sir made us do earlier last week. Overall, the activity was done successfully with all the output in place, and the counting working properly.	

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: None

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: None

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: None

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.



I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.